

Moving the bot from a local PC to a Windows VPS — step by step

This is the real, tested process for taking an already-working `bot/` setup off your local machine and onto a Windows VPS so it runs 24/7 without your PC. It reflects what actually worked (and what didn't) migrating this bot to an IONOS Windows VPS with 3 live MT5 accounts.

Use this either as a manual checklist, or paste it to an AI assistant with RDP/SSH access to the VPS and ask it to work through the steps.

0. What you need before starting

- A Windows VPS with **RDP access** (IP, username, password) — that's the only access most providers give you by default.
 - Your bot's `bot/.env` from the working local setup (has all account credentials, Telegram tokens, etc).
 - 30–60 minutes; the MT5 broker-login step (\$5) can take longer if the broker is unusual — see the gotcha below.
-

1. Turn on SSH so you're not stuck driving everything through RDP

RDP alone is painful to automate against. Get SSH working first — it only takes a few minutes and then everything else is scriptable.

Via RDP, open **PowerShell as Administrator** (not `cmd.exe` — `cmd` can't run `Add-WindowsCapability`) and run:

```
Add-WindowsCapability -Online -Name OpenSSH.Server~~~~0.0.1.0
Start-Service sshd
Set-Service -Name sshd -StartupType Automatic
```

Set PowerShell (not `cmd`) as the default shell for SSH sessions — makes every future command much less painful:

```
reg add "HKLM\SOFTWARE\OpenSSH" /v DefaultShell /t REG_SZ /d "C:\Windows\System32\WindowsPowerShell\powershell.exe"
net stop sshd
net start sshd
```

Gotcha: cloud providers (IONOS and others) often run a *second*, separate firewall layer in their web control panel, distinct from Windows Firewall. Windows Firewall alone being open isn't enough — you

also need an inbound rule for TCP port 22 in the provider's own panel (usually under something like "Firewall Policies" → Incoming rules). Leaving the "Allowed IP" field blank means "allow all IPs," not literally typing "all" (that throws an "Invalid IP address" error). Give it a couple of minutes to propagate after saving.

2. Drive it from your side with paramiko

Don't hand-type commands over an interactive SSH session. Use a small reusable Python helper so every command and its output round-trips cleanly:

```
import paramiko

def run(command: str, timeout: int = 60):
    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    client.connect(HOST, username=USER, password=PASSWORD, timeout=15)
    stdin, stdout, stderr = client.exec_command(command, timeout=timeout)
    print(stdout.read().decode(errors="replace"))
    print(stderr.read().decode(errors="replace"))
    client.close()
```

And a matching SFTP helper for file transfer. Keep both — you'll use them constantly for the rest of this process.

If invoking `run()` from a bash tool/wrapper, wrap the PowerShell command string in *single* quotes at the bash level if it contains `$` — bash expands `$_`, `$something` etc. before it ever reaches PowerShell, silently corrupting the command.

3. Install Python and the bot's dependencies

```
# Install Python 3.11+ manually via RDP (winget/choco if available), then:
cd C:\TradingBot
pip install -r bot\requirements.txt
pip install "numpy<2"      # MetaTrader5 package breaks on NumPy 2.x
pip install requests      # bot/mt5_bridge.py needs it but it's missing from requirements.txt
pip install tzdata        # Windows has no bundled IANA timezone data
```

4. Upload the bot folder — but never blindly overwrite `.env`

Upload everything **except** skip `__pycache__` and `.pyc` files. The first upload should include `.env` (so credentials exist at all), but:

After the first upload, the VPS's own `.env` becomes the source of truth and diverges from your local copy — specifically because the one-time Telegram login (§6) writes `TG_SESSION_STRING` directly into the VPS's `.env`, and your local copy never gets that value back. From then on, **only upload specific changed `.py` files**, never the whole folder, or you will silently wipe the Telegram session and have to redo the OTP login.

```
FILES = [  
    (r"local\path\bot\accounts.py", "C:/TradingBot/bot/accounts.py"),  
    # ...only the files you actually changed  
]
```

5. Install MT5 — one fully isolated terminal per account

If the bot trades multiple MT5 accounts, **do not** point them all at one shared terminal install. The MT5 Python API can only hold one live connection per process, and multiple terminal instances sharing one install directory fight over the same local control port (`127.0.0.1:22346`, `bind error`).

Install the terminal separately per account, e.g.:

```
C:\MT5-AccountA\terminal64.exe  
C:\MT5-AccountB\terminal64.exe  
C:\MT5-AccountC\terminal64.exe
```

Each gets its own MetaTrader5 installer run, pointed at a different install folder. Confirm no stray `terminal64.exe` processes are left over between attempts:

```
taskkill /F /IM terminal64.exe
```

Known gotcha: some brokers never resolve via "Login to Trade Account"

For white-label/prop-firm brokers, the standard **File** → **Login to Trade Account** → **search box** can fail to resolve the broker/server no matter what you try (Python API, portable mode, unattended `/config:` login, session injection) — repeatedly timing out with `(-10005, 'IPC timeout')` or `(-10001, 'IPC send failed')`.

Fix: use the *other* MT5 login flow instead — **File** → **Open an Account** → **search by the broker's company name** → **"Connect with an existing trade account"** → **pick the correct server from the dropdown** (double check you're not picking a similarly-named demo/trial variant, e.g. `BrokerName-Real` vs `BrokerName-Experience`). This one-time GUI step (via RDP) caches the broker/server info, after which the automated Python `mt5.initialize()` call works normally from then on.

6. One-time Telegram login (interactive OTP)

The bot's Telegram session needs a live OTP code from your phone the first time. If you don't have a way to do interactive stdin over your remote tooling, bridge it: run the login script on the VPS in the background, poll a small marker file for the "enter code" prompt, and relay the OTP from wherever you can read it (Telegram itself, on your phone). Once `TG_SESSION_STRING` is populated in the VPS's `bot/.env`, this step never needs repeating (see the `.env` warning in §4 — don't overwrite it later).

7. Run it as a real Windows service, not a console window

A console window tied to your RDP session dies when you disconnect. Use NSSM (Non-Sucking Service Manager) instead:

```
nssm install TradingBot "C:\Path\To\python.exe" "-m bot.main"
nssm set TradingBot AppDirectory "C:\TradingBot"
nssm set TradingBot AppStdout "C:\TradingBot\logs\bot_stdout.log"
nssm set TradingBot AppStderr "C:\TradingBot\logs\bot_stderr.log"
nssm start TradingBot
sc config TradingBot start= auto
```

Now it survives RDP disconnects, and auto-starts on VPS reboot.

8. Give yourself a way to see it's actually working

Two small desktop shortcuts (created via `WScript.Shell` COM in PowerShell, saved to `[Environment]::GetFolderPath('Desktop')` — **not** a hardcoded `C:\Users\<name>\Desktop` path, which can be wrong on Windows Server) make day-to-day checks trivial without needing SSH each time:

- **Start Trading Bot** → runs `net start TradingBot`
- **View Bot Logs** → opens a console live-tailing the log: `powershell -NoProfile -Command "Get-Content '<log path>' -Wait -Tail 50"`

If the bot already has a Telegram command interface (check for something like `/status` in its listener code), that's an even easier way to check status from your phone without RDP at all.

9. Verify, then cut over

1. Confirm the service is `Running` and set to `Auto` start.
2. Watch the log (or `/status`) for all accounts reporting connected and the strategies you expect running.

3. Only once the VPS is confirmed live: **stop every bot process on the local PC** (check for stragglers — it's easy to end up with duplicate `bot.main` processes across full-mode and `--api`-only invocations, which silently double-executes trades). Confirm nothing bot-related is still running locally before considering the migration complete.
 4. Your local PC can now be turned off — an RDP/SSH session disconnecting does **not** stop anything on the VPS; server-side processes are independent of any remote viewer window.
-

Rules that are easy to get wrong here

- **Never store the VPS password in the registry** (e.g. `Winlogon` auto-logon) for convenience — treat this the same as any other production credential.
- **Never re-upload the whole `bot/` folder** once the VPS has its own live `.env` state (Telegram session, per-account halt files) — diff and push only what changed.
- **Don't spawn a second MT5 Python connection against a live terminal** just to check data — the API only supports one connection per process, and killing `terminal64.exe` to "free it up" will disconnect the live bot. Prefer reading the bot's own logs, or its own HTTP API, over a fresh MT5 session.
- A disconnected RDP/SSH session is not a stopped bot — don't re-verify this fact by restarting anything "just to be sure."